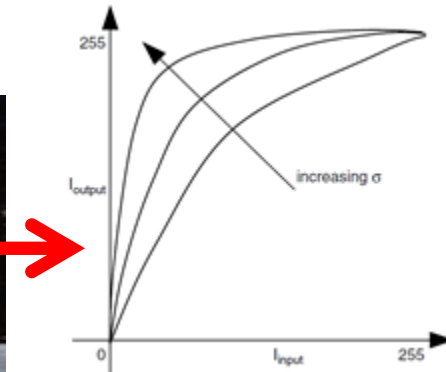


Image Processing - Lecture 3



Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

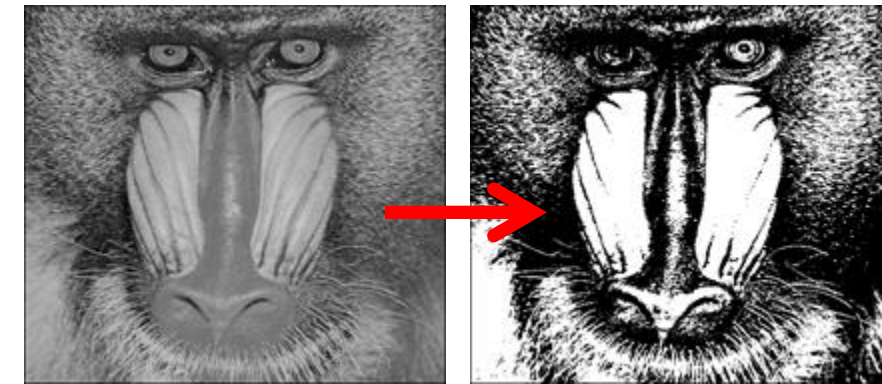
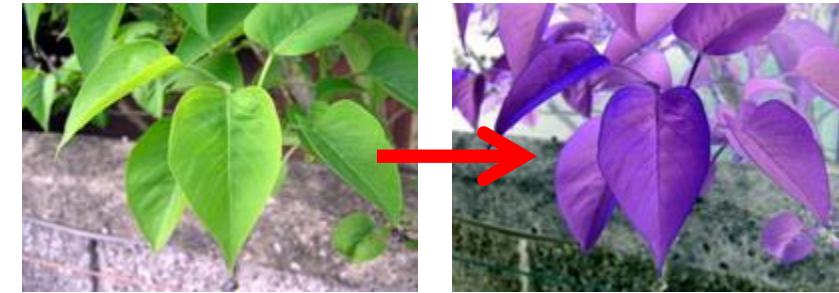
Reading:

Szeliski – Section 3.1.1 (very brief)

Gonzalez/Woods – Section 3.2.2 / 3.2.3

Point Intensity Transforms for Contrast Enhancement

Recap ... Point Transforms



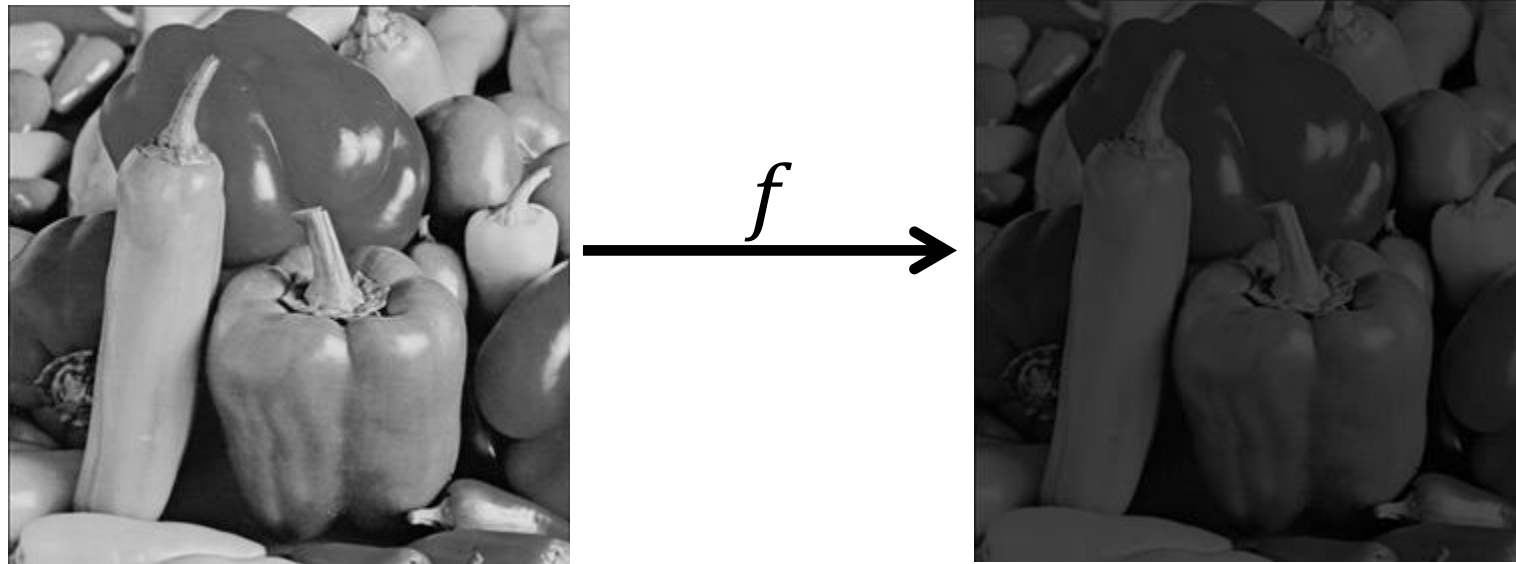
Lecture 2: arithmetic & logical operations

Today: Functional Point Transforms

Processing each pixel value, p , individually using a mathematical function

$$p' = f(p)$$

as a point transform operator transforms the image from one state to another.



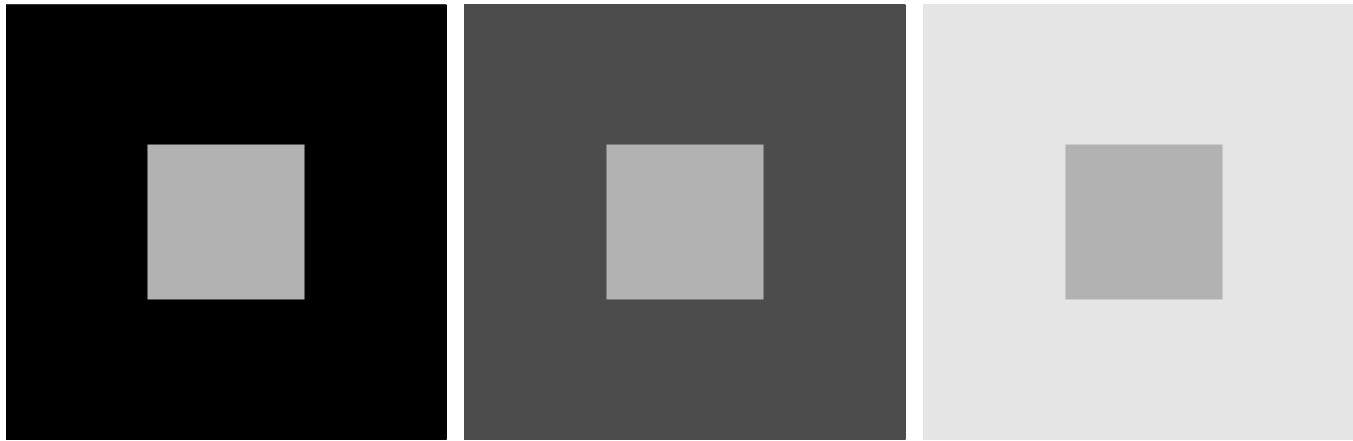
*transforms image from one state to another – also known as **intensity transform functions***

Main application: **Image Enhancement**

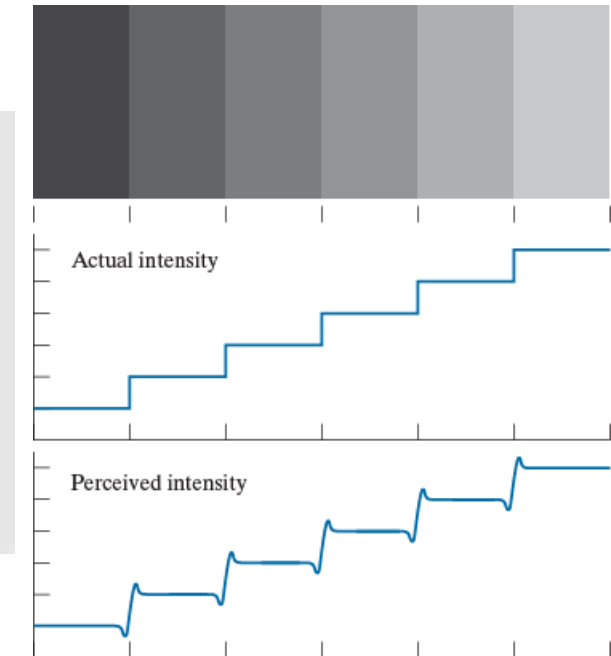
Image Enhancement

- **Goal:** make the image look better
 - view and process visual information with greater clarity.
- Image enhancement is **subjective**. It depends on:
 - the **information required** by the visual task
 - the **physical characteristics** of the image
 - the user's **prior knowledge/experiences**
 - the user's **intuition and judgement**
- **Evaluation methodology:** **perceived quality** of results.
(certain enhancement methods for computer vision applications, such as vehicle number plate recognition, can be objectively measured.)

Image Enhancement for Human Visual System



All inner squares have the same intensity, but they appear darker as the background becomes lighter.



Mach band effect – Perceived intensity is not a simple function of actual intensity.

Common poor image characteristics:
poor lighting and **noise**

This lecture focuses on issues of
poor lighting (illumination)

image noise removal:
later in the course

Dynamic Range

- **Dynamic range of a sensor, display or image** is the largest (possible) signal value divided by the smallest (possible) signal value.
- The range of a sensor is the **set of all possible intensity values** of the images it can capture.
- A “*good*” image should utilise the full (or most of the) of the sensor’s range.
- Increasing the dynamic range improves contrast.

Important notes on pixel value processing:

- Can be floats in the range $[0, 1]$ (no worries for rounding errors, etc.)
- Can be integers - e.g. in the range $[0, 255]$ for 8-bit images
 - Integer processing may have decimal places during processing which we round to the closest value.

High Dynamic Range Processing

In photography, dynamic range is measured in “*stops*”. An increase of one stop represents doubling the amount of light.



Images captured at different dynamic ranges can be combined to create “tone mapped” images.
[beyond the scope of this lecture]



Images from Wikipedia

Logarithmic Transform

- **Operation:** logarithmic transform replaces each pixel value with its logarithm:

$$I_{output}(i, j) = \log I_{input}(i, j)$$

- compresses dynamic range of image (i.e. smaller range of pixel values)
 - maps a narrow range of **low intensity values** in the input, into a wider range of output levels
- In practice, we control the range using the function:

$$I_{output}(i, j) = c \cdot \log[1 + (e^{\sigma} - 1)I_{input}(i, j)]$$

with scaling parameters σ and c .

Logarithmic Transform

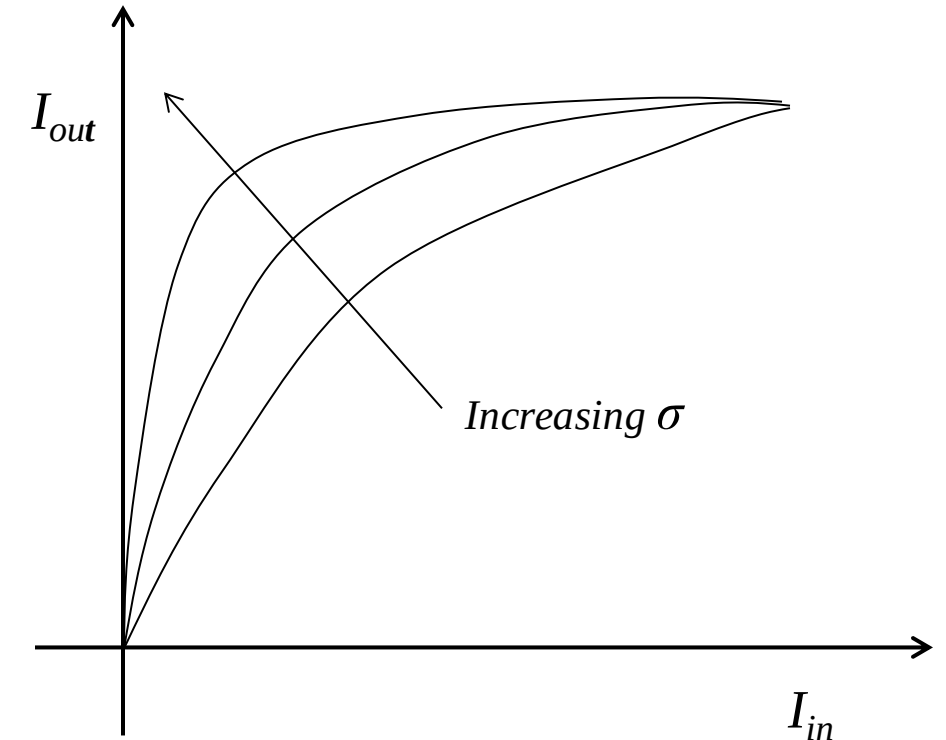
- σ controls the range of the values onto which the logarithmic function is applied.
- c normalises the output to the range $[0,255]$:

$$c = \frac{255}{\log(1 + |R|)}$$

- where R is the maximum of value in

$$I_{input}(i, j)$$

- Increasing σ allocates a larger part of the output range to low intensity values.



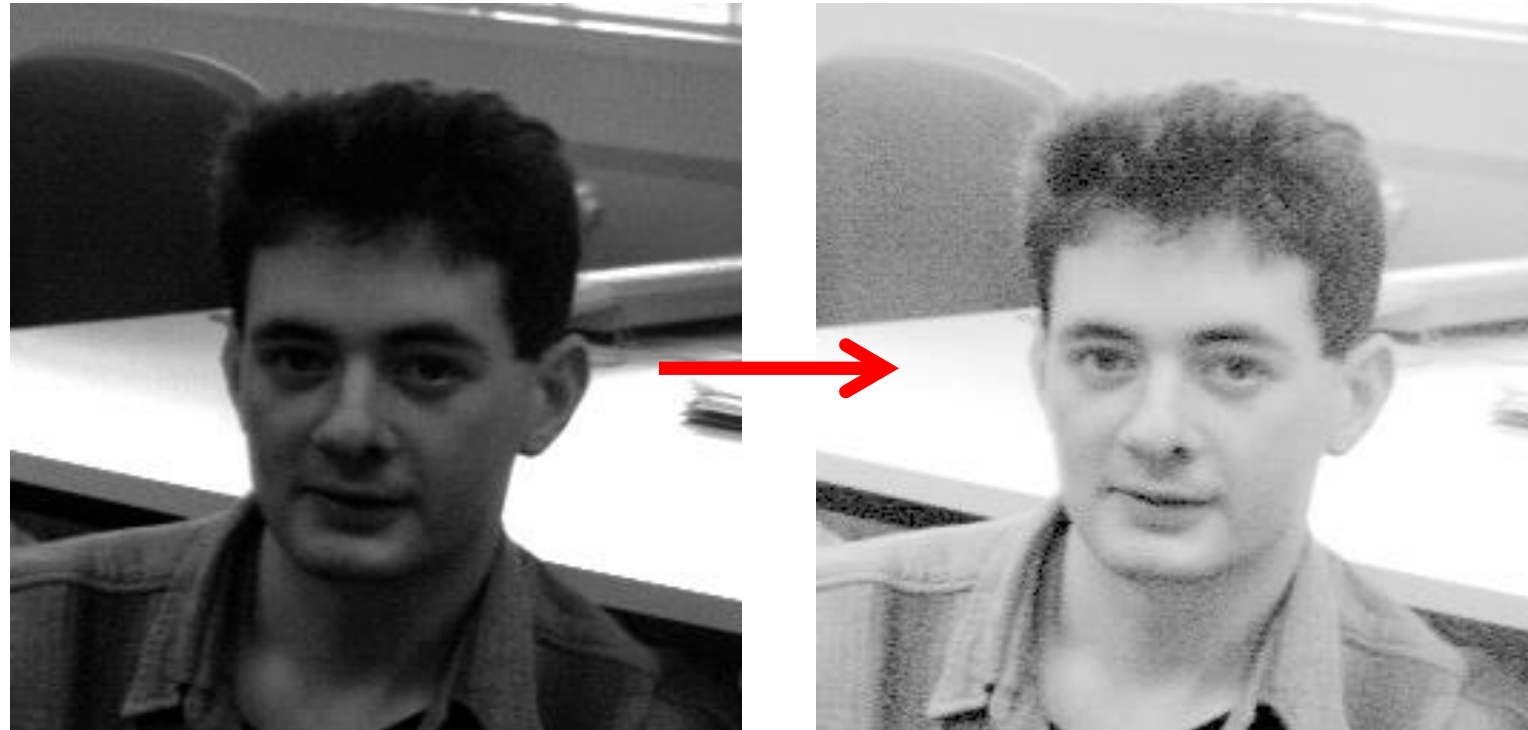
Logarithmic Transform

- The dynamic range of the sensor/camera was too small to capture the scene well.
(dark foreground – bright background)



- As a result of a (usually automatic) decision on the camera exposure, the dynamic range of the dark parts was compressed.

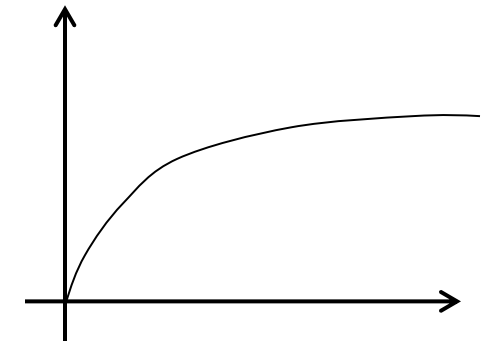
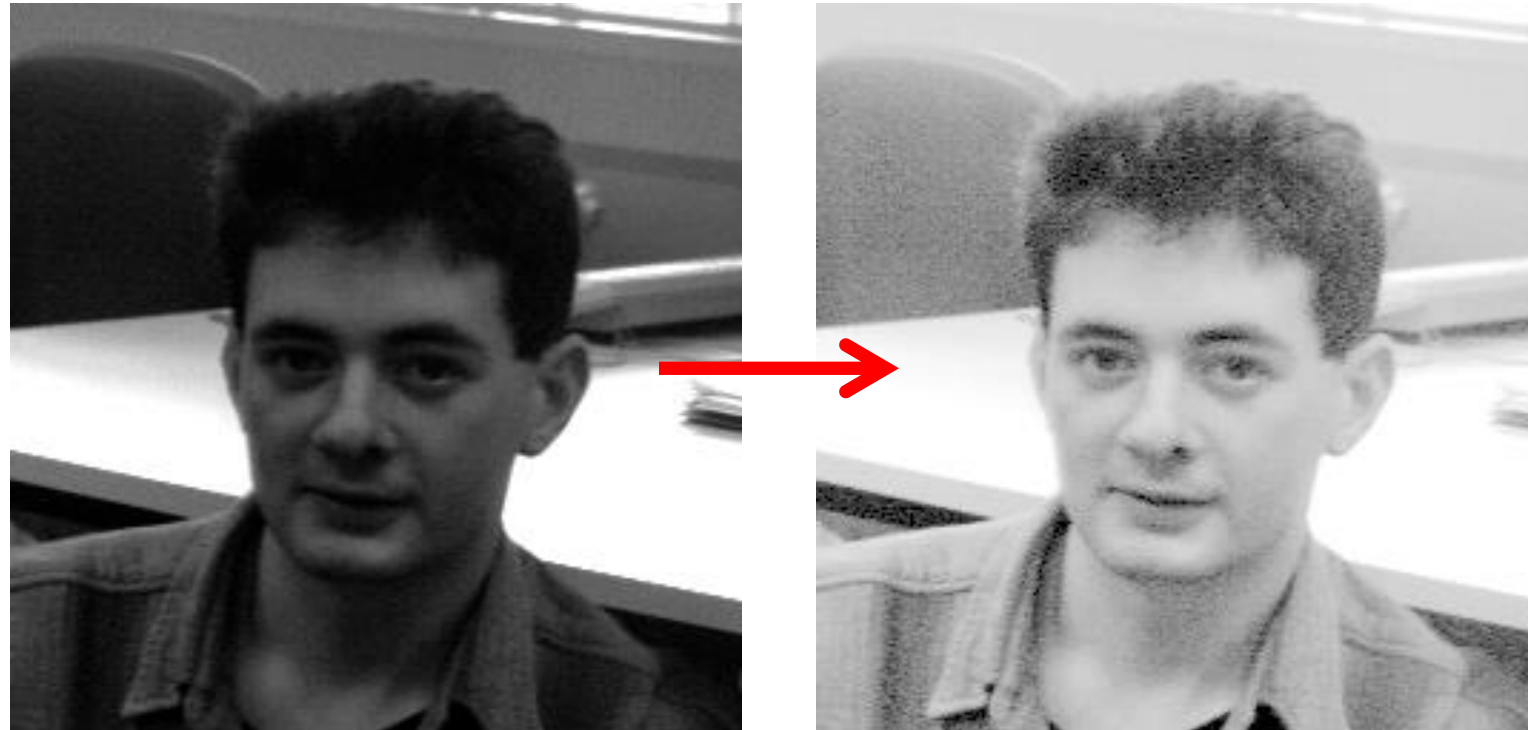
Logarithmic Transform



The logarithmic transform in this example:

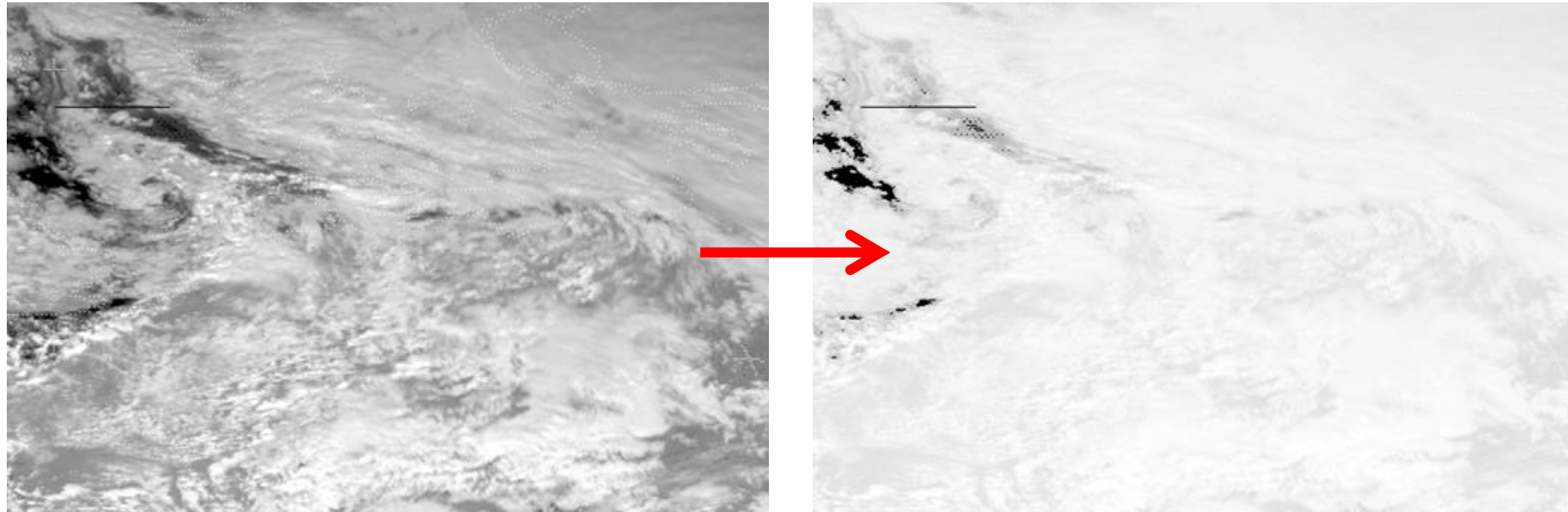
- **Brightens** the foreground (**dark pixels**), spreading low pixels values over a wider range (also spends more of the output dynamic range budget...).
- **Compresses** the background pixel range (the **bright high value pixels**).

Logarithmic Transform



- The logarithmic transform
 - **increased** the dynamic range of the **dark** parts of the image
 - **decreased** the dynamic range of the **bright** parts of the image.
- Spread out the low values (dark) and compressed the high values (bright).

Logarithmic Transform



- In this example, most of the information is in the high value (bright) range.
- Applying the logarithmic transform yields **poor results**
 - information loss and worse visualisation.

Application: X-Ray Images

- Sensors return X-ray intensity as an exponent:

$$I(i, j) = I_o \cdot \exp(-f(i, j))$$

- I_o = the X-ray source intensity.
- f = material attenuation properties (object thickness and material density).

If linearly scaled these down to 8-bits, bright pixels would dominate

- The logarithmic transform cancels the exponent:

$$\log I_{out} = \log[I_o \cdot (\exp(-f(i, j)))]$$

- As a result, the image gives a linear mapping of the material properties.



Often embedded into X-ray imaging processes

Quick Demonstration

- Logarithmic Transform:



https://github.com/atapour/ip-python-opencv/blob/main/logarithmic_transform.py

Exponential Transform

- **Operation: exponential transform** is the inverse of the logarithmic transform.
 - Compresses dynamic range differently
 - Replaces each pixel value with its **exponent**:

$$I_{output}(i,j) = \exp(I_{input}(i,j))$$

- In practice, we use a variable basis and scaling:

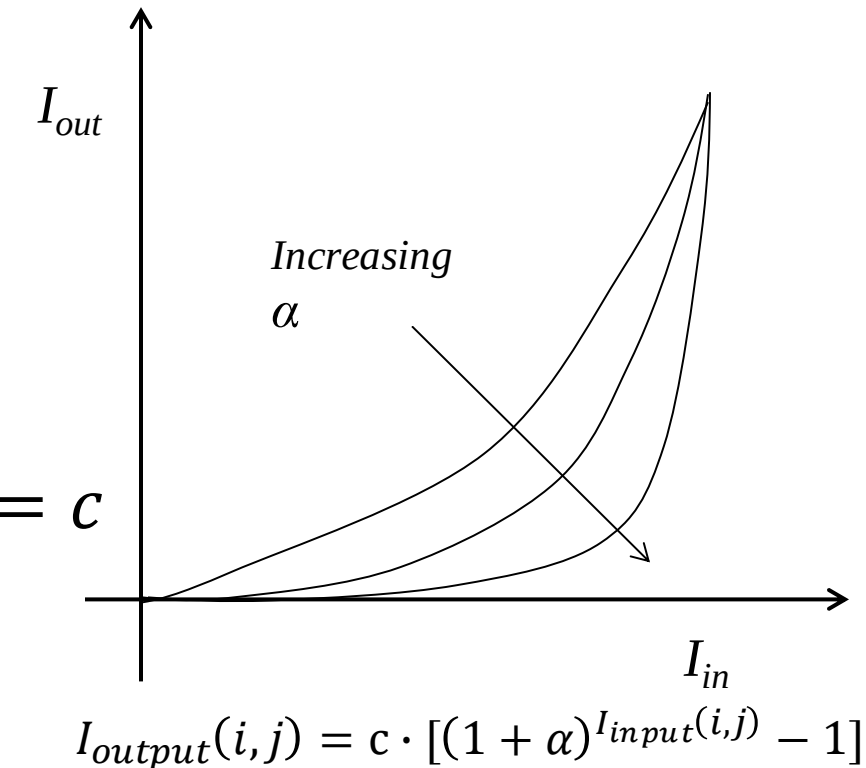
$$I_{output}(i,j) = c \cdot [(1 + \alpha)^{I_{input}(i,j)} - 1]$$

where $1 + \alpha$ is the basis and c the scaling factor.

Exponential Transform

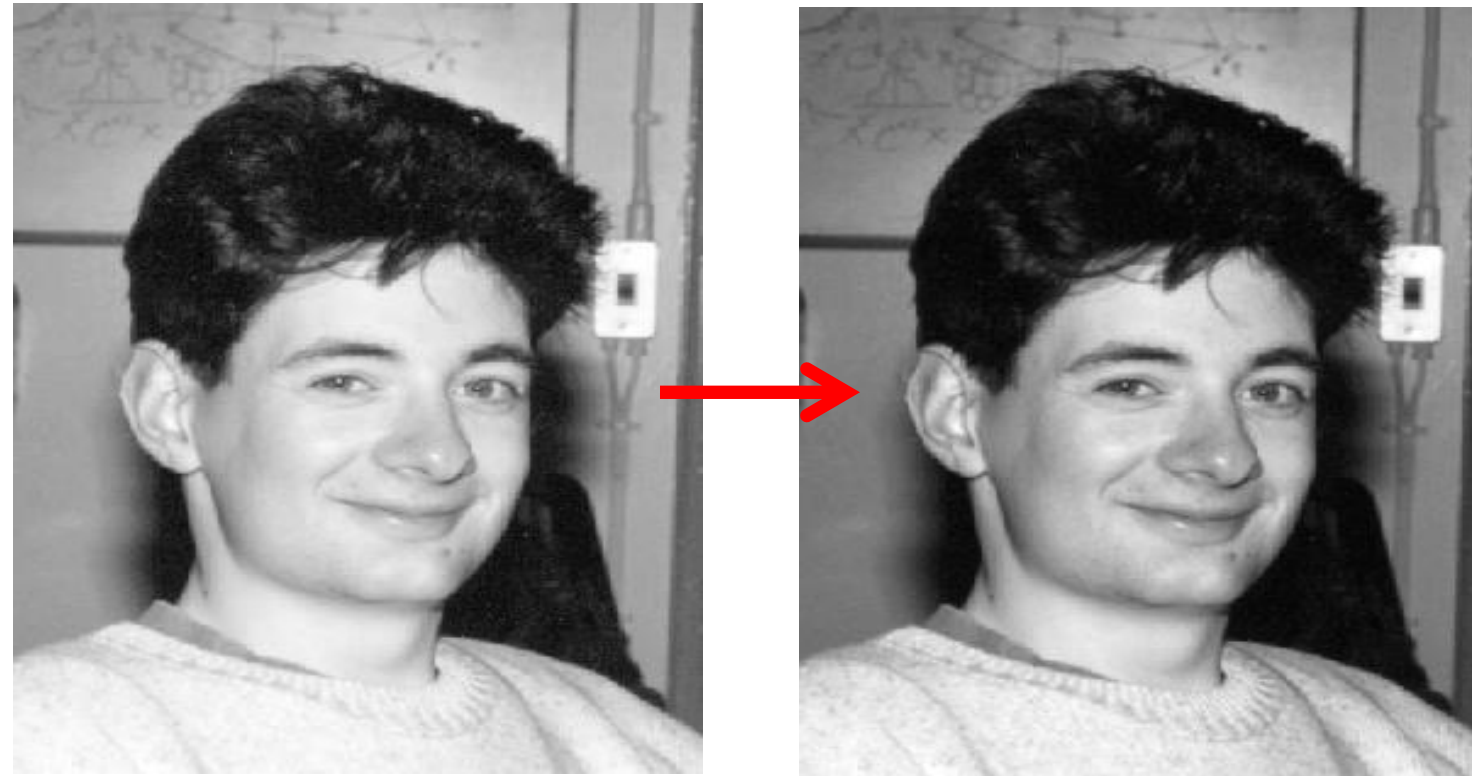
- $1 + \alpha = \text{basis}$
- $c = \text{scaling factor}$

- $I_{input}(i, j) = 0$ gives $I_{output}(i, j) = c \cdot [(1 + \alpha)^{I_{input}(i, j)}] = c$
 - we subtract 1 to prevent offset in output.



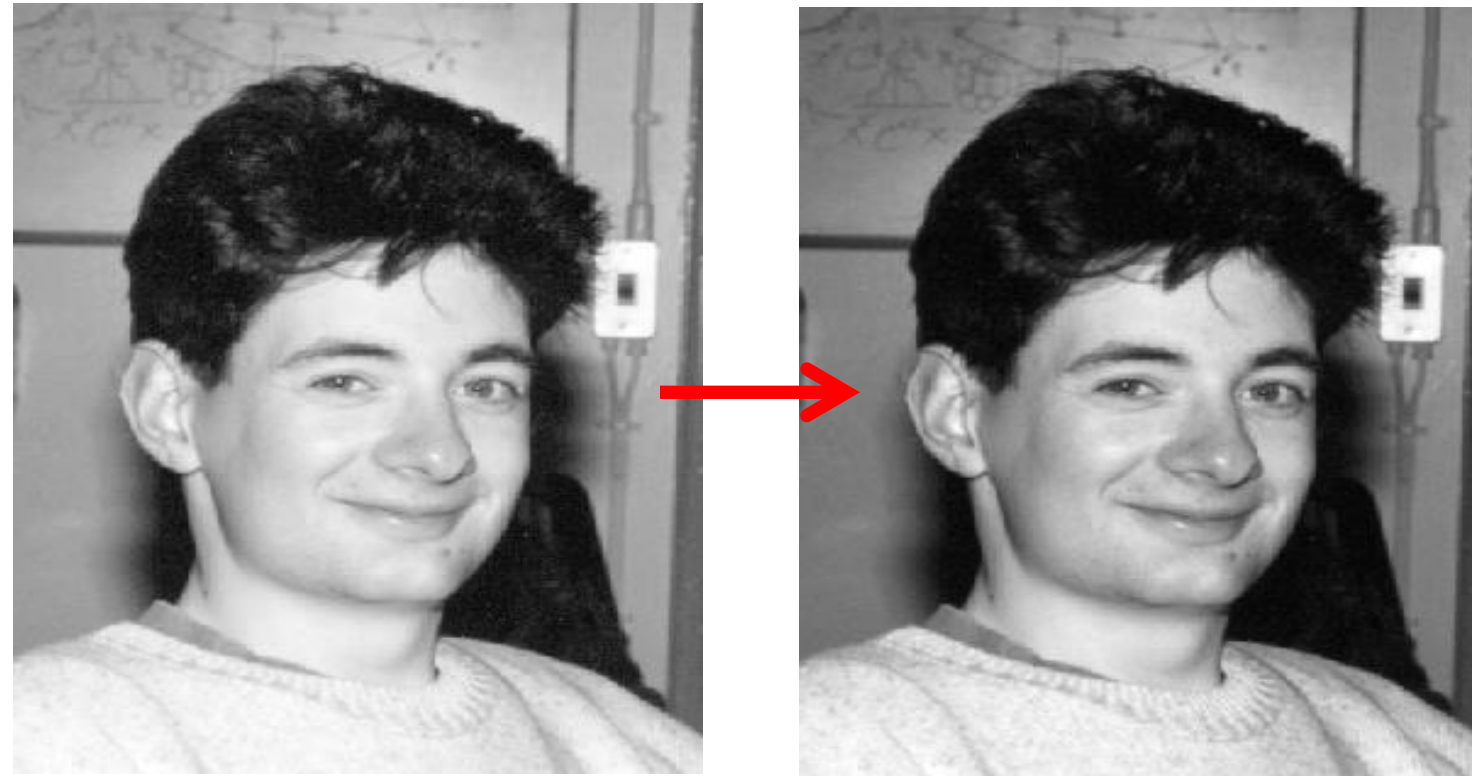
- Basis > 1 is suitable for photographic image enhancement (decrease the dynamic range of dark regions - increase the dynamic range of bright regions).

Exponential Transform



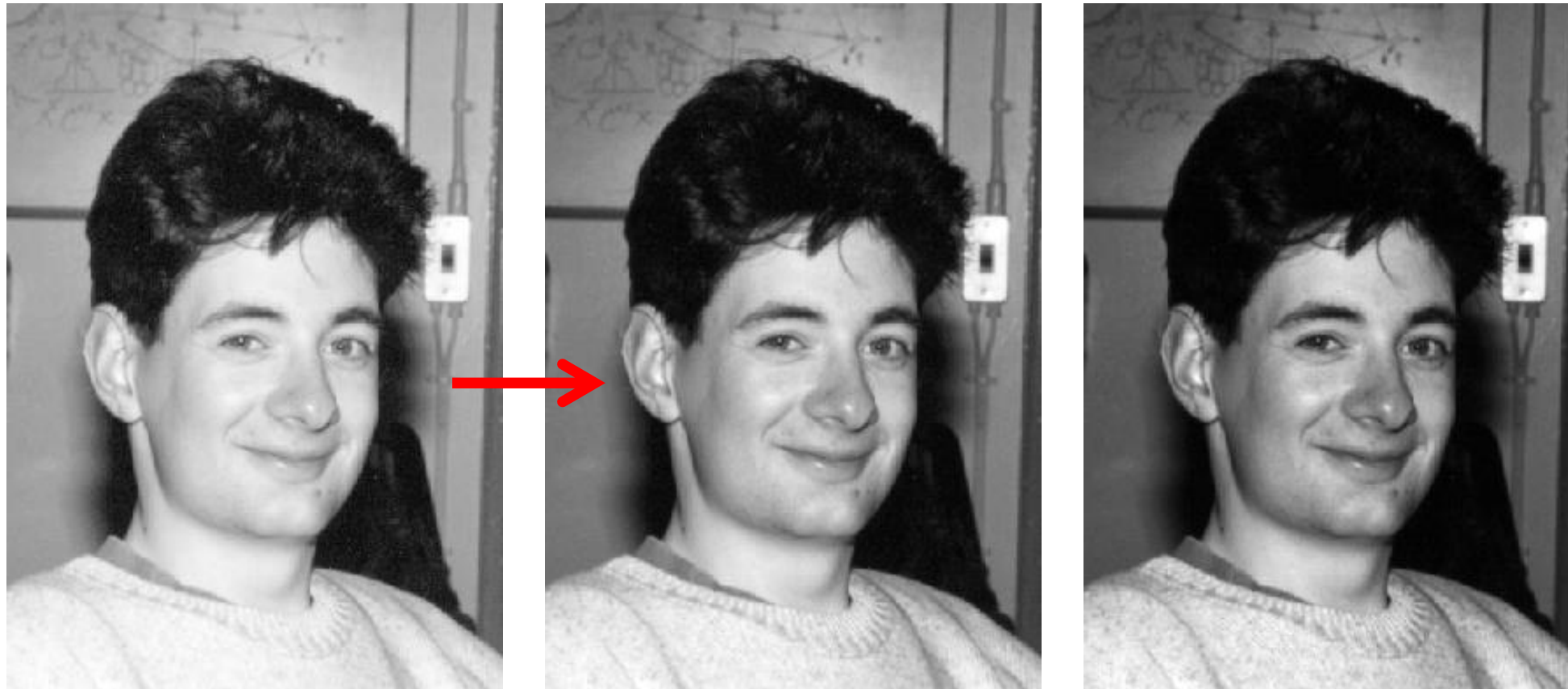
- The **exponential transform** **decreases** the dynamic range of **dark regions** whilst **increasing** the dynamic range in **light regions**.
- Enhances **detail** in high value (**bright**) regions.

Exponential Transform



Important:

- Information was already there; we did not generate anything new!
- Transforms make information more readily available to the human visual system.



base = 1.005

base = 1.01

- In this example, face = high valued (bright) / hair = low valued (dark).
- The **face contrast improved** at the expense of the contrast in the darker areas of the image (e.g. hair).

Quick Demonstration

- Exponential Transform:



https://github.com/atapour/ip-python-opencv/blob/main/exponential_transform.py

Power-Law ('raise to power') Transform

- **Operation:** Raise each pixel value to a fixed power:

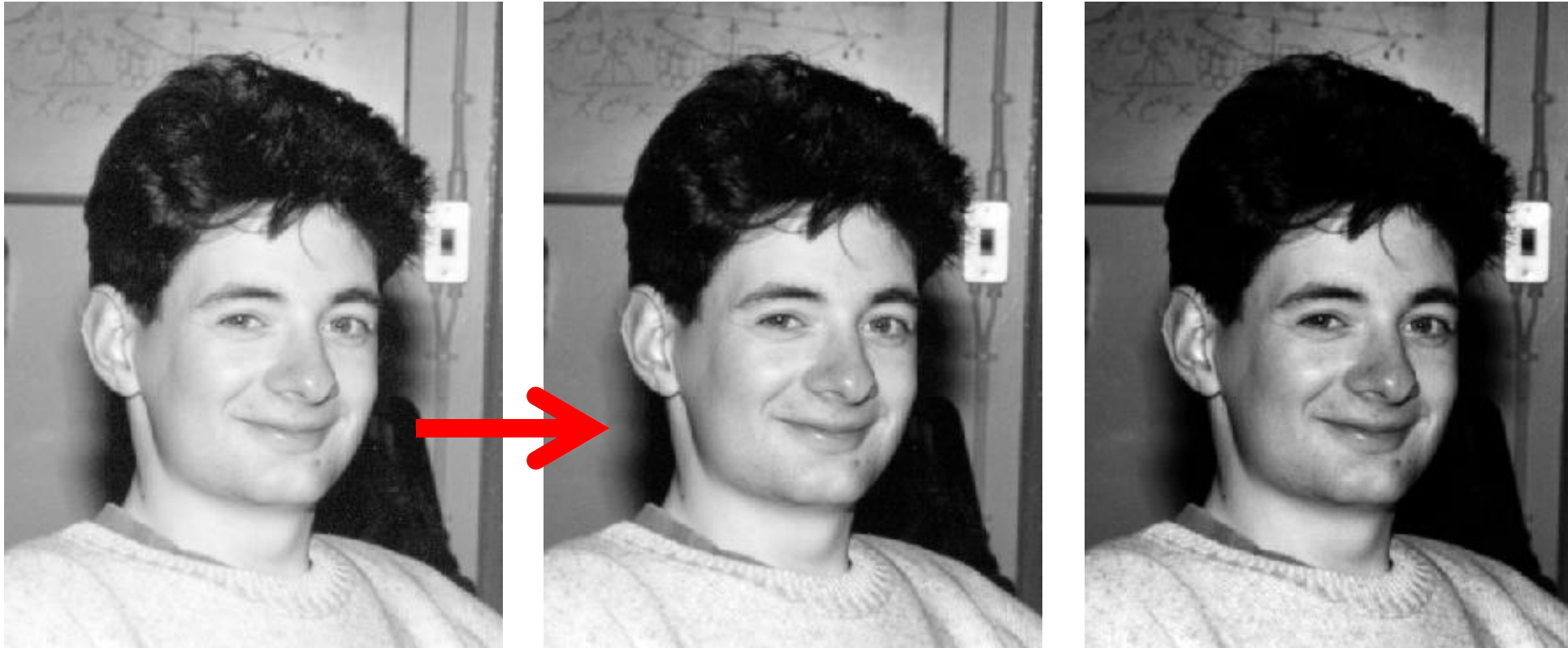
$$I_{output}(i, j) = c \cdot (I_{input}(i, j))^r$$

varying r

Notice the input is on the
base not the exponent

- For $r > 1$, it **enhances** (spreads) **high-value intensities**, compresses low-value intensities.
- For $r < 1$, it **enhances** (spreads) **low-value intensities**, compresses high-value intensities.
- The “power-law” transform has similar effect to the **logarithmic (when $r < 1$)** and or to the **exponential (when $r > 1$)**.
 - In practice, the input image, I_{input} , is **normalised to [0,1]** for the operation and then back to [0,255] for visualisation.

Power-Law Transform - Example



source: HIPR2 © 2003/4

$r = 1.5$

$r = 2.5$

- $r > 1$ enhances high value pixels at the expense of low value pixels.
- In this example: **face/board** contrast **improved**; **hair/eye** contrast **worse**.

Application: Gamma Correction

- ❑ Nonlinear operation used to encode luminance values in image systems.
- ❑ Originally designed to compensate for input/output characteristic of CRTs.
- ❑ r is traditionally called the **gamma value**, and the process **gamma correction**.
- $f(x) = x^\gamma$ with $\gamma < 1$ weights the intensities toward higher (**brighter**) values.
 - An underexposed photo can be gamma corrected with $\gamma < 1$.
- $f(x) = x^\gamma$ with $\gamma > 1$ weights the intensities toward lower (**darker**) values.
 - An overexposed photo can be gamma corrected with $\gamma > 1$.
- $f(x) = x^\gamma$ with $\gamma = 1$ has no effect on the image.

Gamma Correction

Original Overexposed Photo



$$\gamma = 1$$



$$\gamma = 2$$



$$\gamma = 3$$



$$\gamma = 4$$

Corrected Photo

Gamma Correction

Original Underexposed Photo



$$\gamma = 1$$



$$\gamma = 1/2$$



$$\gamma = 1/3$$



$$\gamma = 1/4$$

Corrected Photo

Quick Demonstration

- Power Law Transform (Gamma Correction):

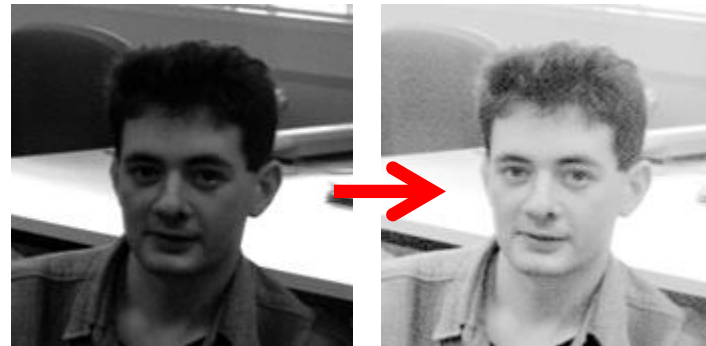


https://github.com/atapour/ip-python-opencv/blob/main/gamma_correction.py

Summary

Functional point transforms:

- logarithmic transform
- exponential transform
- gamma correction



Application: contrast enhancement.

